

# Fichier sale : 15 colonnes diagnostiquées une à une



Avant d'écrire la moindre requête, repère ce que chaque colonne cache vraiment.

Un test technique de Data Analyst ne note pas la requête la plus élégante : il regarde ce que tu vois dans le fichier avant même d'ouvrir ton éditeur. Ce guide diagnostique 15 colonnes d'un fichier volontairement sale, colonne par colonne, avec le bon outil pour chacune. Ni Excel ni SQL ni pandas ne gagne à tous les coups : le bon réflexe, c'est de savoir lequel choisir selon l'anomalie. Cinq familles de 3 colonnes, un diagnostic complet par colonne : anomalie, cause, geste, outil.

Un test technique ne se joue pas à la requête que tu écris en dernier. Il se joue avant, au moment où tu ouvres le fichier et où tu regardes vraiment ce qu'il contient. Ce guide ne nettoie rien pour toi : il t'apprend à repérer, colonne par colonne, l'anomalie, sa cause probable, le geste qui la confirme et l'outil qui va le plus vite. Excel, SQL et pandas se partagent les 15 colonnes à parité : aucun des trois ne gagne partout, et savoir lequel choisir fait partie de la compétence évaluée.

## Colonnes temporelles.

*Le premier piège classique d'un test technique : des dates qui se lisent à l'œil, mais qui ne se trient pas.*

### 1. date\_inscription FORMATS MÉLANGÉS

**Anomalie :** trois écritures différentes dans une même colonne, chacune lisible isolément : 14/03/2023, 2023-03-15 et 03-14-2023.

```
14/03/2023
2023-03-15
03-14-2023
```

#### Pourquoi ça casse

Une règle de parsing unique échoue sur au moins deux lignes sur trois, sans lever d'erreur bloquante : le tri ou le filtre chronologique se casse en silence.

**Le geste :** en Excel, colonne auxiliaire =ESTNUM(DATEVALUE(A2)) pour isoler les lignes non reconnues. En SQL, un CAST ou un TRY\_CONVERT avec un format explicite, jamais une conversion implicite.

**Outil recommandé :** Excel pour le repérage visuel rapide sur petit fichier, SQL si le contrôle doit être rejoué à l'échelle.

### 2. date\_maj

**Anomalie :** une ligne datée du 31/12/2026, postérieure à la date du jour.

```
SELECT *
FROM clients
WHERE date_maj > CURRENT_DATE;
```

#### Pourquoi ça casse

Une date de mise à jour dans le futur n'est jamais une donnée valide, c'est une erreur de saisie ou de fuseau, et elle fausse tout calcul d'ancienneté fait dessus.

**Le geste :** la requête ci-dessus isole ces lignes avant tout calcul, pas après.

**Outil recommandé :** SQL, contrôle à rejouer en amont de chaque pipeline.

### 3. anciennete\_calculée

**Anomalie :** une colonne d'ancienneté en jours qui ne bouge plus depuis le dernier export.

```
dates = pd.to_datetime(df["date_inscription"], format="mixed", dayfirst=True)
df["anciennete_jours"] = (pd.Timestamp.today() - dates).dt.days
```

#### Pourquoi ça casse

Calculée une fois puis stockée en valeur, la colonne se désynchronise du reste du fichier dès que la date système avance.

**Le geste :** convertir la colonne en date puis recalculer, jamais faire confiance à la valeur stockée, comme ci-dessus.

**Outil recommandé :** pandas, recalcul vectorisé et reproductible à chaque exécution.

#### À retenir, palier 1

Le format d'affichage d'une date ne dit rien de son type réel.

Avant tout tri ou tout filtre chronologique, contrôle le type réel de la colonne, pas seulement ce que l'œil y lit. Une colonne qui *ressemble* à une date n'en est pas forcément une pour le moteur qui va la trier.

## Colonnes texte et catégorielles.

*Ici, l'œil et la base ne voient pas la même chose : une différence visuelle n'est pas toujours une différence de valeur.*

### 4. categorie CASSE INCOHÉRENTE

**Anomalie :** Vip, VIP et vip coexistent dans la même colonne.

```
SELECT LOWER(TRIM(categorie)) AS cat_norm, COUNT(*) AS n
FROM ventes
GROUP BY LOWER(TRIM(categorie));
-- Vip / VIP / vip : 3 lignes distinctes avant normalisation, 1 seule après
```

#### Pourquoi ça casse

Un GROUP BY brut crée trois groupes distincts pour une seule vraie catégorie, et fausse tout comptage par segment sans qu'aucune requête ne plante.

**Le geste :** normaliser avant de compter, avec la requête ci-dessus. En Excel, =NOMPROPRE(A2) uniformise la casse à l'écriture.

**Outil recommandé :** SQL pour objectiver le volume réel, Excel pour la correction ponctuelle.

### 5. pays

**Anomalie :** FR et France désignent le même pays, sans convention commune dans la colonne.

```
mapping = {"FR": "France", "fr": "France", "France": "France",
          "DE": "Allemagne", "Allemagne": "Allemagne"}
df["pays_norm"] = df["pays"].map(mapping)
```

#### Pourquoi ça casse

Une jointure ou une agrégation géographique dessus échoue silencieusement sur une partie des lignes, sans erreur qui remonte.

**Le geste :** construire une table de correspondance et l'appliquer avec .map(), comme ci-dessus.

**Outil recommandé :** pandas, la table de correspondance est réutilisable sur tout futur import.

## 6. commentaire

**Anomalie :** des accents transformés en séquences illisibles, sans motif régulier apparent.

```
"café terminé" mal réimporté devient "cafÃ© terminÃ©"  
pd.read_csv("export.csv", encoding="utf-8")
```

### Pourquoi ça casse

Un mauvais encodage à l'import (latin-1 lu comme UTF-8, ou l'inverse) casse les caractères accentués sans lever d'erreur bloquante.

**Le geste :** réimporter en fixant l'encodage, jamais corriger caractère par caractère à la main.

**Outil recommandé :** pandas ou Excel selon la source, toujours au réimport.

### À retenir, palier 2

Ce que tu vois n'est pas toujours ce que la base compare.

Une valeur texte visuellement différente n'est pas forcément une valeur différente pour le moteur qui la regroupe. Normalise avant de compter, jamais après.

## Colonnes numériques et calculées.

*Un total qui tombe juste rassure trop vite : il ne prouve rien sur la propreté de la colonne qui l'alimente.*

## 7. montant TEXTE DÉGUISÉ EN NOMBRE

**Anomalie :** des valeurs comme 1234,56, alignées à droite en apparence mais stockées en texte.

```
SELECT montant, CAST(REPLACE(montant, ',', '.')) AS REAL) AS montant_num  
FROM facture;  
-- "1234,56" (texte) devient 1234.56 (nombre)
```

### Pourquoi ça casse

Un SUM ou un AVG dessus est faux, ou échoue selon le moteur, sans que la colonne paraisse anormale à l'œil.

**Le geste :** convertir avant tout calcul, avec REPLACE puis CAST en SQL. En Excel, =VALEUR(SUBSTITUE(A2;",";".")).

**Outil recommandé :** SQL si la colonne alimente un pipeline, Excel pour un contrôle ponctuel.

## 8. montant\_ttc

**Anomalie :** quelques lignes avec un montant TTC négatif, sans mention d'avoir.

```
SELECT * FROM facture WHERE montant_ttc < 0;
```

### Pourquoi ça casse

Hors avoir documenté, un montant TTC négatif signale une anomalie de saisie ou d'import, pas une donnée métier valide.

**Le geste :** lancer la requête d'audit ci-dessus avant tout calcul de chiffre d'affaires.

**Outil recommandé :** SQL, contrôle à rejouer à chaque nouvel import.

## 9. quantite TYPE MIXTE SILENCIEUX

**Anomalie :** des valeurs comme dix ou N/A mélangées à des entiers dans la même colonne.

```
df["quantite_num"] = pd.to_numeric(df["quantite"], errors="coerce")
lignes_fautives = df[df["quantite_num"].isna()]
```

#### Pourquoi ça casse

Une colonne qui semble numérique casse un calcul sans erreur explicite si chaque valeur n'est pas testée individuellement.

**Le geste :** `pd.to_numeric` avec `errors="coerce"` transforme l'illisible en NaN, et isole nommément les lignes fautives.

**Outil recommandé :** pandas, seul outil qui liste précisément les lignes en échec de conversion.

#### À retenir, palier 3

Un total qui tombe juste ne prouve pas que la colonne est propre.

Une somme correcte peut cacher des erreurs qui s'annulent, ou des lignes silencieusement ignorées par le calcul. Teste la colonne, pas seulement son résultat agrégé.

## Identifiants et doublons.

*Un identifiant qui paraît unique à l'œil ne l'est pas forcément pour la base qui le compare caractère par caractère.*

### 10. id\_client DOUBLON INVISIBLE

**Anomalie :** C-1024 et c-1024 (espace final) traités comme deux clients différents.

```
SELECT LOWER(TRIM(id_client)) AS id_norm, COUNT(*) AS n
FROM clients
GROUP BY LOWER(TRIM(id_client))
HAVING COUNT(*) > 1;
-- "C-1024" et "c-1024 " comptent pour 2 identifiants avant normalisation
```

#### Pourquoi ça casse

Un espace ou une casse différente crée un doublon qu'un `GROUP BY` brut sur `id_client` ne détecte jamais.

**Le geste :** normaliser avec `TRIM` et `LOWER` avant de grouper, puis filtrer avec `HAVING COUNT(*) > 1`.

**Outil recommandé :** SQL, seule requête qui objective le volume réel de doublons cachés.

### 11. email

**Anomalie :** des lignes vides mélangées à des doublons exacts dans la même colonne.

```
SELECT COUNT(*) AS total_lignes,
       COUNT(NULLIF(email, '')) AS lignes_avec_valeur,
       COUNT(DISTINCT NULLIF(email, '')) AS emails_uniques
FROM clients;
```

#### Pourquoi ça casse

`COUNT(*)` seul surestime la qualité de la colonne en noyant les vides dans le total.

**Le geste :** comparer côte à côte lignes renseignées, doublons et valeurs uniques, comme dans la requête ci-dessus.

**Outil recommandé :** SQL pour le diagnostic chiffré, Excel (valeurs en double) pour un contrôle visuel rapide.

### 12. telephone

**Anomalie :** 06 12 34 56 78, +33612345678 et 06.12.34.56.80 pour des numéros comparables.

```
digits = re.sub(r"\D", "", telephone)
if digits.startswith("0033"):
    digits = "0" + digits[4:]
elif digits.startswith("33") and len(digits) == 11:
    digits = "0" + digits[2:]
```

#### Pourquoi ça casse

Une déduplication ou un rapprochement échoue tant que la colonne n'est pas ramenée à un format unique.

**Le geste :** ne garder que les chiffres par expression régulière, puis réécrire l'indicatif, comme ci-dessus.

**Outil recommandé :** pandas, seul outil pratique pour appliquer une règle de normalisation en masse.

#### À retenir, palier 4

Un identifiant unique en apparence ne l'est pas forcément en base.

Espace, casse, format d'écriture : tout ce que l'œil ignore, la base le compare au caractère près. Normalise avant de compter des doublons, jamais après.

## Colonnes structurelles.

*Sur le terrain, un fichier de test technique ressemble presque toujours trait pour trait à un export de production dégradé, pas à un cas d'école inventé pour l'occasion.*

### 13. code\_postal SPÉCIFIQUE EXCEL

**Anomalie :** 01000 devient 1000 après ouverture dans Excel.

```
=SI(NBCAR(A2)<5;"Anomalie";"OK")
```

#### Pourquoi ça casse

Excel supprime par défaut les zéros en tête d'une colonne formatée en nombre, ce qui casse tout code postal commençant par 0, sans avertissement.

**Le geste :** forcer la colonne en format Texte avant l'import, puis contrôler la longueur avec la formule ci-dessus.

**Outil recommandé :** Excel, le problème lui est propre : la prévention à l'import prime sur la correction après coup.

### 14. statut VALEUR ORPHELINE

**Anomalie :** une valeur résilié alors que le référentiel métier ne connaît qu'actif, inactif et archive.

```
SELECT c.statut, COUNT(*) AS n
FROM clients c
LEFT JOIN ref_statut r ON LOWER(TRIM(c.statut)) = r.statut
WHERE r.statut IS NULL
GROUP BY c.statut;
```

#### Pourquoi ça casse

Une typo ou un ancien libellé fausse tout comptage par statut sans lever d'erreur, la valeur reste silencieusement hors référentiel.

**Le geste :** isoler les orphelins par LEFT JOIN contre la table de référence avant tout comptage, comme ci-dessus.

**Outil recommandé :** SQL, seule requête qui objective l'écart au référentiel.

### 15. colonne\_fantome

**Anomalie :** une colonne générique, style `champ_x`, dont personne dans l'équipe ne connaît l'origine.

```
df["champ_x"].value_counts(dropna=False)
```

### Pourquoi ça casse

Supprimer sur intuition est un risque ; sans objectiver son usage réel, tu peux effacer une donnée qu'un autre process consomme discrètement.

**Le geste** : regarder la distribution réelle avec `value_counts(dropna=False)` avant toute décision de suppression.

**Outil recommandé** : pandas, une ligne de code suffit à trancher la question à la place de l'intuition.

### À retenir, palier 5

Regarde avant d'écrire, sur les 15 colonnes comme sur les prochaines.

Une colonne générique sans usage documenté se questionne, elle ne se supprime pas sur intuition. Le réflexe qui vaut pour `code_postal`, `statut` et `colonne_fantome` vaut pour tout fichier que tu n'as pas construit toi-même.

15 colonnes, 15 diagnostics, trois outils. Tu n'as pas besoin de mémoriser chaque geste au mot près. Retiens la posture : avant d'écrire une requête ou une formule, regarde ce que la colonne cache vraiment. C'est cette lecture-là, plus que le SQL parfait, qui distingue un candidat qu'on retient d'un candidat qui a récité un tutoriel.

## Fait main, en solo.

Si ce guide t'a aidé, partage-le autour de toi : il peut débloquent un autre candidat. Et si tu veux que je traite un sujet précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

**Dylan**

